Practical Course: Accelerating Convolutional Neural Networks using Programmable Logic

Project: CNN Inference

Group: 1 Date of Submission: 18.07.2026

Nr.	Matr. Nr.	Name
1	03827739	Moser Stefan
2	03799351	Sezgin Ulas

1 Introduction

Convolutional neural networks achieve high classification accuracy but require substantial computation and memory access, making efficient inference challenging on embedded systems. This project accelerates a pre-trained CIFAR-10 CNN using the programmable logic of the PYNQ-Z2 platform. The design exploits parallelism, data reuse, on-chip buffering, and fixed-point arithmetic, with a particular focus on the computationally dominant convolutional layers. The accelerator is evaluated in terms of classification accuracy, execution time, resource utilization, and speedup over the ARM implementation.

2 Background

This chapter introduces CNN inference, the target network, and the computational characteristics that motivate its hardware acceleration.

2.1 CNN Inference and Convolution

During inference, a CNN uses pre-trained weights to transform an input into a class prediction. Most of its computational workload is concentrated in convolutional layers, which extract features by applying multiple filters to the input feature maps.

As shown in Figure 1, an input contains C channels of size $H \times W$. Each of the M filters spans all input channels and produces one output feature map. An output activation is computed as

$$O[m, y, x] = b[m] + \sum_{c=0}^{C-1} \sum_{r=0}^{R-1} \sum_{s=0}^{S-1} W[m, c, r, s]\, I[c, y + r - p, x + s - p], \tag{1}$$

where p denotes the zero padding. The target network uses 3×3 filters, unit stride, and $p = 1$, preserving the spatial dimensions. Each convolution is followed by ReLU activation.

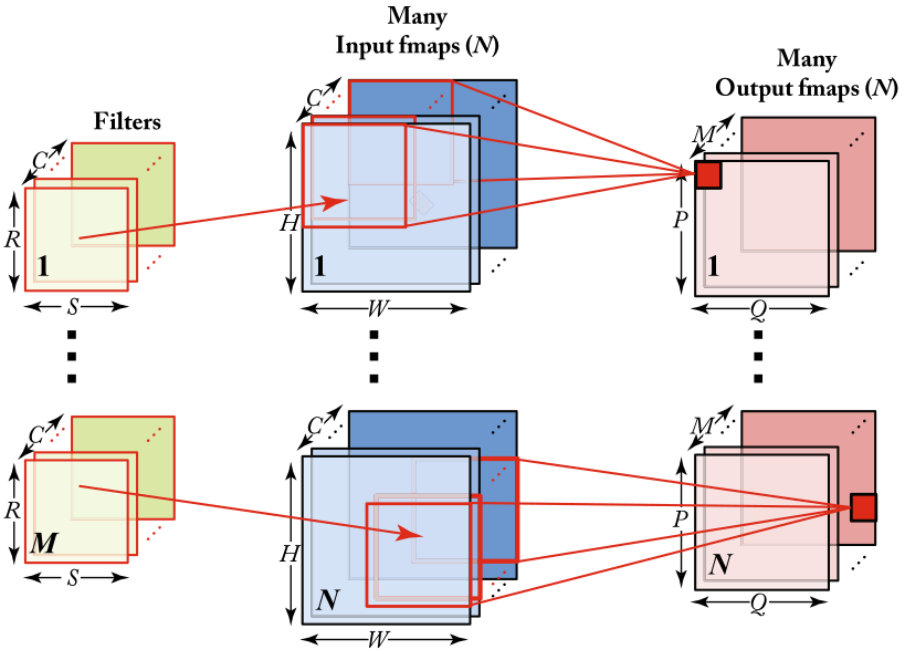


Figure 1: High-dimensional convolution in a CNN. Each filter spans all input channels and generates one output feature map. Reproduced from [?].

2.2 Target CNN Model

The target CNN classifies normalized $3 \times 32 \times 32$ CIFAR-10 images into ten classes. It contains 19 convolutional layers arranged in three stages. Two max-pooling layers reduce the spatial resolution while the channel count increases from 16 to 32 and 64. Global average pooling, a linear layer, and softmax form the classifier. The reference model achieves approximately 89% test accuracy.

Table 1: Architecture of the target CNN model.

Stage	Operation	Output size
Input	RGB image	$3 \times 32 \times 32$
Initial	Conv–ReLU	$16 \times 32 \times 32$
Block 1	$6\times$ Conv–ReLU	$16 \times 32 \times 32$
Pool 1	2×2 Max Pool	$16 \times 16 \times 16$
Block 2	$6\times$ Conv–ReLU	$32 \times 16 \times 16$
Pool 2	2×2 Max Pool	$32 \times 8 \times 8$
Block 3	$6\times$ Conv–ReLU	$64 \times 8 \times 8$
Classifier	Global Avg. Pool $\rightarrow$ Linear $\rightarrow$ Softmax	$64 \times 1 \times 1 \rightarrow 10 \rightarrow 10$

2.3 Computational Characteristics

The number of multiply-accumulate operations (MACs) required by a convolutional layer is given by

$$N_{\mathrm{MAC}} = H_{\mathrm{out}} W_{\mathrm{out}} C_{\mathrm{out}} C_{\mathrm{in}} K^2. \tag{2}$$

The 19 convolutional layers of the target network require approximately 40.55 million MACs for a single image, whereas the fully connected layer requires only $64 \times 10 = 640$ MACs. Therefore, convolution accounts for nearly the entire computational workload. In addition, convolution requires frequent access to weights, input activations, and partial results, making data movement another important performance factor. These characteristics motivate the parallelism and data-reuse techniques employed in the accelerator.

3 Design and Implementation

The accelerator partitions CNN inference between the ARM processor and the programmable logic. The computationally intensive convolutional stages, including ReLU, max pooling, and global average pooling, are executed in the hardware kernel. The final linear and softmax layers are evaluated in software using the 64 features returned by the accelerator. This section describes the system partitioning, convolution engine, and memory organization of the design.

3.1 Top-level Architecture

Figure 2 illustrates the top-level architecture and hardware–software partitioning of the inference implementation.

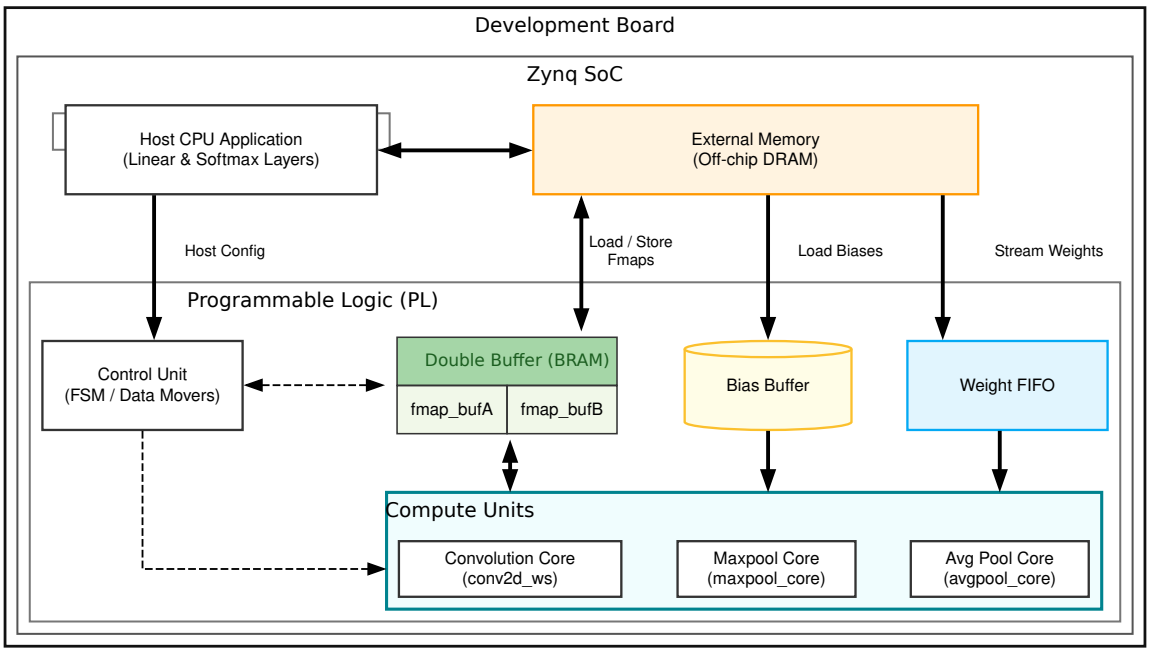


Figure 2: System-level architecture and hardware–software partitioning of the CNN inference implementation.

The host application prepares the inputs and rearranges the convolution weights into the layout expected by the accelerator. The weights and biases are transferred to external memory once and reused for subsequent input images. For each image, the FPGA accelerator executes the 19 convolution and ReLU layers, two max-pooling layers, and global average pooling. The control unit coordinates the whole process by selecting the respective compute unit: *Convolution Core*, *Maxpool Core* or *Avg Pool Core* for the current layer and directing the dataflow accordingly. Input images and intermediate feature maps are stored in on-chip BRAM buffers. To enable efficient processing without memory conflicts, we implemented a double-buffering scheme in which inputs are read from *fmap_bufA* while outputs are written to *fmap_bufB*. The buffers are then switched for processing the next layer. Due to their limited size, we also store the CNN's biases completely in BRAM. Weights are streamed over an intermediate FIFO to the compute unit (for more details see Subsection 3.2) After iterating through all the layers the resulting 64-element feature vector is returned to the processor. The ARM processor then executes the linear layer, softmax, and argmax to obtain the predicted class. Separate AXI master interfaces are used for the input, weights, biases, and output, while the kernel is controlled through an AXI4-Lite interface.

3.2 Convolution Core

A single convolution core is reused across all 19 convolutional layers. The implemented approach processes N output channels at once. Since the number of output channels varies between 16, 32, and 64, they are processed in tiles of $T_{\text{out}} = 16$. For each input channel in a tile, the engine loads a set of $16 \times 3 \times 3$ weights. The weights are arranged such that one 512-bit memory word contains the 16 weights associated with the same kernel coordinate. Consequently, only nine packed words are required to load the complete 3×3 kernels for one input channel and output-channel tile. A FIFO connects the weight-loading stage to the convolution core, allowing both stages to operate concurrently.

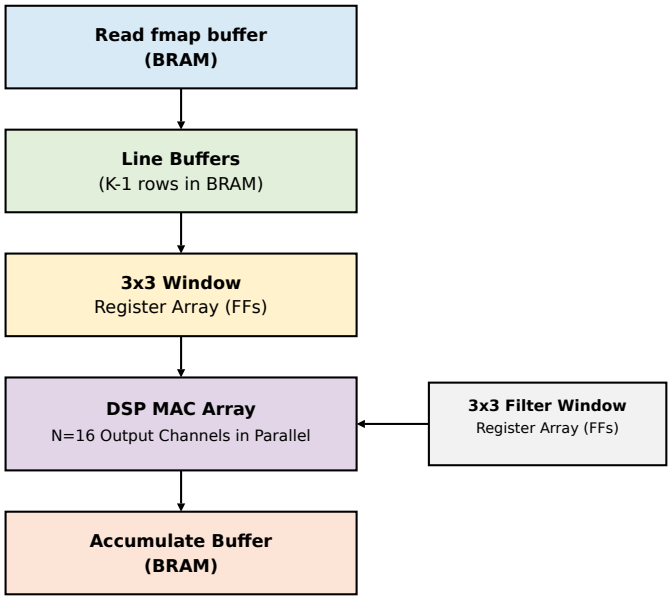


Figure 3: Convolutional Core Dataflow.

The overall flow of the convolutional core is depicted in Figure 3. Input activations are read from the on-chip feature-map buffer into a line buffer. The line buffer always retains the previous two rows, while a fully partitioned 3×3 register window is always being updated and used for efficient computation. Zero padding is generated through boundary checks, avoiding the creation of a separate padded feature map. The spatial processing loop is pipelined with an achieved initiation interval of one. The kernel loops are fully unrolled, enabling the nine products for 16 output channels to be evaluated at once, completely in parallel. This results in $16 \times 9 = 144$ parallel multiplications for each input window. The contributions from successive input channels are accumulated in 16 independent partial-sum buffers. After all input channels have been processed, RELU is applied to the accumulated values and they written to the output feature-map buffer. Layers with more than 16 output channels are completed by processing additional output-channel tiles with the same convolution engine.

4 Evaluation

4.1 Simulation Results

After synthesizing and simulating our design in Vitis, the reports showed a total execution time of 589,598 cycles and an estimated Fmax of 175.10 MHz. Running the board at 150MHz we would expect an kernel runtime of about 3.9ms. The reported resource utilization is listed in Table 2. As one can see the design fits quite comfortably on the board.

Table 2: PYNQ-Z2 Resource Utilization Overview

Resource Type	Your Values	Board Maximum	Utilization (%)
Block RAM (BRAM)	93	140	66.43%
DSP Slices	153	220	69.55%
LUT (Look-Up Tables)	30,647	53,200	57.61%
FF (Flip-Flops)	33,249	106,400	31.25%

4.2 Board Measurements

We deployed our design on the Pynq-Z2 board, tested the FPGA-implementation for a 2000 image benchmark at multiple clock frequencies and compared against the CPU reference in Table 3 and Figure 4. **Note:** For some reason the board never ran at the frequency specified in the .cfg files, but always ran at a $\times 1.2$ speed of the FCLK0. So we had to force the boards clock FCLK0 manually, resulting in the frequencies below. Interestingly the design was also capable of running at 180MHz. As one can see from the results, the design was cabable of processing 222 Images/s for the 150MHz design, whith only a slight reduction of 0.15% in accuracy due to quantization. Comparing the total runtimes, this design achieved a $\times 16.70$ speedup compared to the AMD Ryzen CPU and a $\times 129.3$ speedup compared to the ARM CPU. The average write, read and cpu time per image was 0.045ms, 0.019ms and 0.019ms respectively, making up only a small fraction of the whole computation.

Table 3: Performance comparison of CPU and FPGA implementations (2000 image benchmark).

Platform	Acc. (%)	Total (s)	Images/s	Kernel/Image (ms)
ARM CPU	90.30	1165.08	1.7166	–
AMD Ryzen	90.30	150.47	13.90	–
120 MHz	90.15	10.87	184.05	4.9067
150 MHz	90.15	9.01	222.00	3.9695
180 MHz	90.15	8.05	248.46	3.5020

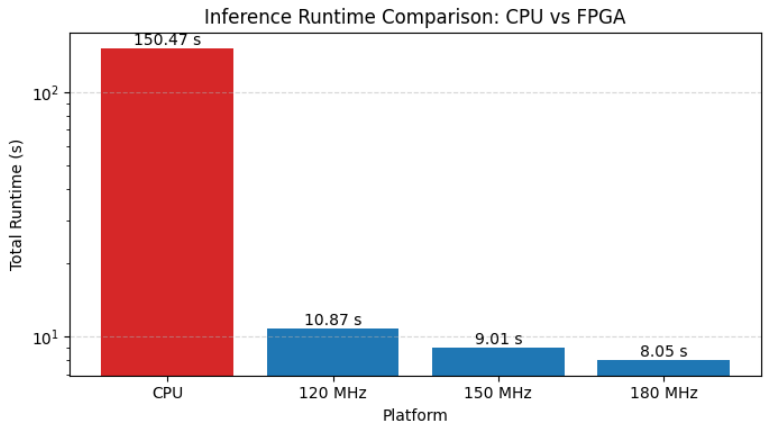


Figure 4: Runtime comparison AMD Ryzen CPU vs FPGA.

5 Conclusion

While the measurements show quite promising results there is still allot of room for potential improvements in our design. The convolution core for example, spends allot of extra cycles on reading and writing data. This excess time could be entirely removed by executing those steps in parallel while the computation is happening. For example, instead of a separate writeback at the end of each tile, we could split the task more intelligently and introduce *DATAFLOW* pragmas, allowing writeback as soon as the results have arrived. Additionally, the weights and activations of the next input channel could already be loaded seperately while the current input channel is still being processed. Furthermore, we could try to quantize the inputs before they are being loaded to the DRAM to allow data to be streamed more efficiently.